

Università degli Studi di Perugia



Dipartimento di Matematica e Informatica

Progetto – SDEP

F1

Studenti

Rossi Riccardo

Matricola

357175

Anno Accademico 2024-2025

Capitolo 1. Descrizione generale del progetto

Il progetto sviluppato consiste nella realizzazione di un servizio web distribuito che permette la gestione e l'acquisto di biglietti per le gare del campionato mondiale di Formula 1. L'intero sistema è stato pensato per soddisfare i requisiti del cliente. In particolare prevedendo la progettazione e implementazione di un **Web Service REST/JSON** con interazione su più livelli (client, server, database).

L'applicazione è suddivisa in due componenti principali:

- **Backend RESTful** sviluppato con Node.js e Express.js
- **Frontend** realizzato in HTML5, CSS3 e JavaScript puro
- **Database MySQL**, che gestisce la persistenza dei dati relativi a utenti, biglietti, gare e classifiche

Il sistema prevede due tipi distinti di utenti:

- **Fruitori**: utenti finali registrati che possono consultare il calendario delle gare, visualizzare i biglietti disponibili e procedere all'acquisto
- **Erogatori**: amministratori che hanno il compito di gestire il sistema, modificando la disponibilità e i prezzi dei biglietti, o cancellando interi settori di vendita.

Capitolo 2. Architettura del Web Service

L'architettura è costruita secondo il paradigma RESTful. Le interazioni tra client e server avvengono tramite chiamate HTTP che restituiscono e accettano dati in formato JSON.

2.1 Tecnologie Utilizzate:

- **Node.js + Express.js**: per la gestione della REST
- **MySQL**: per l'archiviazione dei dati
- **JavaScript** (frontend): per l'interattività lato client
- **Postman** (per testare REST)

2.2 Struttura degli endpoint REST:

Autenticazione e Registrazione

- **POST /register**: Permette all'utente di registrarsi inviando nome, cognome, email, password, conferma password e codice carta d'identità. I dati vengono validati e salvati nel database.
- **POST /login**: Autentica l'utente restituendo un token di sessione se le credenziali sono corrette. Gli utenti con ruolo "admin" vengono reindirizzati all'interfaccia di amministrazione.

Gestione Biglietti (Utente)

- **GET /biglietti/:idGara**: Ritorna i settori disponibili per una gara specifica, con dettagli su prezzo e quantità residua.
- **POST/acquista**: Riceve i dati dell'acquisto (id utente, id gara, settore, quantità, codice carta). Verifica disponibilità e aggiorna il database in caso di successo. Restituisce messaggi chiari di conferma o errore.

Gestione Biglietti (Admin)

- **GET /admin/biglietti/:idGara:** Restituisce la lista dettagliata di tutti i settori per una gara, visibile solo agli utenti autenticati come admin.
- **PUT /admin/biglietti:** Aggiorna quantità e/o prezzo di un determinato settore. Richiede payload con id settore, nuova quantità e nuovo prezzo.
- **DELETE /admin/settore/:id:** Elimina un settore dalla gara indicata. L'operazione richiede conferma da parte dell'utente e è accessibile solo ad amministratori.

Esempio flusso REST:

1. L'utente accede a **/login** e ottiene accesso
2. Tramite **GET /biglietti/5**, riceve i biglietti per la gara 5
3. Con **POST /acquista**, effettua un acquisto
4. L'admin modifica i prezzi tramite **PUT /admin/biglietti**

Le chiamate REST sono tutte documentate e possono essere testate con strumenti come Postman. Il server restituisce codici di stato HTTP coerenti (200, 201, 400, 401, 403, 404, 500) accompagnati da messaggi JSON esplicativi.

2.3 Sicurezza e Validazione

- Le operazioni sensibili (acquisto, modifica, eliminazione) richiedono utente autenticato
- I dati in ingresso sono sempre validati sia lato client che lato server
- Sono implementati controlli per evitare SQL Injection e accessi non autorizzati.

Capitolo 3. Ruoli e Funzionalità

Fruitori (Utenti)

- Accesso tramite login e registrazione
- Visualizzazione gare e biglietti disponibili
- Acquisto dei biglietti con inserimento del codice carta (fittizio)
- Visualizzazione dell'area riservata con i propri dati e possibilità di logout

Erogatori (Admin)

- Accesso riservato tramite credenziali predefinite
- Interfaccia di amministrazione per:
 - Visualizzare i settori di ogni circuito
 - Modificare la quantità e il prezzo dei biglietti
 - Eliminare un settore con conferma
- Logout

Capitolo 4. Interazione con il Sistema

L'interfaccia utente è pensata per offrire un'esperienza semplice e responsiva:

- Homepage: storia della Formula 1 con elementi visuali coinvolgenti
- Sezione "Piloti": visualizzazione card interattive dei piloti con dettagli su hover
- Sezione "Scuderie": gallerie di immagini, storia e dettagli tecnici
- Sezione "Calendario": eventi futuri con countdown, link a Google Calendar e, se passati, collegamento ai risultati
- Sezione "News": galleria notizie 3x3 con effetto hover
- Sezione "Classifiche": tabelle dinamiche filtrabili per pilota, scuderia o circuito

L'acquisto dei biglietti avviene tramite una form interattiva con feedback in tempo reale sulla disponibilità e conferma visiva.

Capitolo 5. Esecuzione, Deploy e Test

Requisiti:

- Node.js e npm installati
- MySQL attivo e configurato

Passaggi per avviare il sistema:

1. Configurare il database (file **db.js** con credenziali corrette)
2. Posizionarsi nella cartella del server
3. Eseguire **npm install** per installare le dipendenze
4. Avviare:
 - **node server/server-client.js** per il server utente
 - **node server/server-admin.js** per il server amministratore
5. Accedere a <http://localhost:3000> e <http://localhost:3001> via browser
6. Eseguire test REST con Postman o simili

Conclusioni

Il progetto realizzato soddisfa tutti i requisiti richiesti per il corso di Sistemi Distribuiti e Paralleli:

- Utilizza un'architettura RESTful moderna
- Implementa le principali funzionalità distribuite: autenticazione, autorizzazione, comunicazione client-server, persistenza dati
- Offre un'esperienza utente completa per entrambe le tipologie (fruitori ed erogatori)
- Rispetta le best practice di sicurezza e usabilità

Si configura come una piattaforma robusta e scalabile, che può facilmente essere estesa con nuove funzionalità o integrata con servizi esterni (es. pagamento online reale, invio email, reportistica).